

Programming Languages and Environments (Lecture 18)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Syllabus

- Logic-oriented languages
- Datalog
- Prolog
- Unification
- Backtracking

Datalog

- Datalog is a **declarative** programming language that represents programs using logic.

- A Datalog program comprises a set of rule in the conjunctive form:

$$H \text{ :- } B_1, B_2, \dots B_n \qquad B_1 \wedge B_2 \wedge \dots \wedge B_n \implies H$$

- Predicate **H** is valid (proven) if all predicates **B1** to **Bn** are valid (proven).
- A fact is a rule whose body is empty.

$$\begin{array}{l} \text{Person(John) :- .} \\ \text{A(0) :- .} \end{array} \qquad \text{true} \implies H$$

- Predicates have parameters and an untyped value domain.

Parent(John,Mary), Age(John,30), Person(X)

Datalog

- The program

Parent(John, Mary) :- .

Parent(Mary, Ann) :- .

Ancestor(X, Y) :- Parent(X, Y).

Ancestor(X, Y) :- Parent(X, Z), Ancestor(Z, Y).

- Allows the deduction of the following facts:

Parent(John, Mary)

Parent(Mary, Ann)

Ancestor(John, Mary)

Ancestor(Mary, Ann)

Ancestor(John, Ann)

Program analysis (sketch)

```
i := 10
while i != 0 {
    i := i - 1;
};
return i;
```

```
Block(0, "i := 10") :- .
Block(1, "i != 0") :- .
Block(2, "i := i - 1") :- .
Block(3, "return i") :- .
Pred(0,1) :-
PredT(1,2) :-
PredF(1,3) :-
Pred(2,1) :-
Pred(X,Y) :- PredT(X,Y)
Pred(X,Y) :- PredF(X,Y)
NZero_i(X) :- Block(X, "i:=10")
NZero_i(X) :- PredT(Y,X), Block(Y, "i!=0"), Not_i(X)
Zero_i(X) :- PredF(Y,X), Block(Y, "i!=0"), Not_i(X)
NZero_i(X) :- Pred(X,Z), NZero_i(Z), Not_i(X)
NZero_i(X) :- Pred(X,Z), NZero_i(Z), Not_i(X)

?- Zero_i(3)
```

Datalog

- A Top-down algorithm searches for a proof given a query.
- To search, it needs to use backtracking (upon failing searches, another path).
- A bottom-up approach derives all possible results with a fixed-point algorithm.
- It always terminates because
 - All parameters are ground values or variables (no functors)
 - No negation

Datalog

$A(0,1) :- .$

$A(1,2) :- .$

$A(X,X) :- .$

$A(X,Y) :- A(X, Z), A(Z, Y).$

- Allows the deduction of the following facts:

$A(0,1)$

$A(1,2)$

$A(0,2)$

$A(X,X)$

- And answers positively to the following queries.

$A(0,0)$

$A(1,1)$

$A(99,99)$

Fixed point algorithm

- The (least) fixed-point algorithm allows the implementation of any inductive system (in polynomial time).

```
 $F^0 := \emptyset$   
 $i := 0$   
repetir  
   $F^{i+1} := \emptyset$   
  para cada regra  $H :- B_1, B_2, \dots, B_n \in P$  :  
    se  $B_1, B_2, \dots, B_n \subseteq F^i$  então  $F^{i+1} := F^{i+1} \cup \{H\}$   
     $i := i + 1$   
enquanto  $F^{i+1} = F^i$ 
```


Fixed point algorithm

- The (least) fixed-point algorithm allows the implementation of any inductive system (in polynomial time).

$$F^0 := \emptyset$$
$$i := 0$$

$$F^0 := \emptyset$$

$$F^{i+1} = \{H \mid H \text{ :- } B_1, B_2, \dots, B_n \in P \text{ e, tal que, } B_1, B_2, \dots, B_n \subseteq F^i\}$$

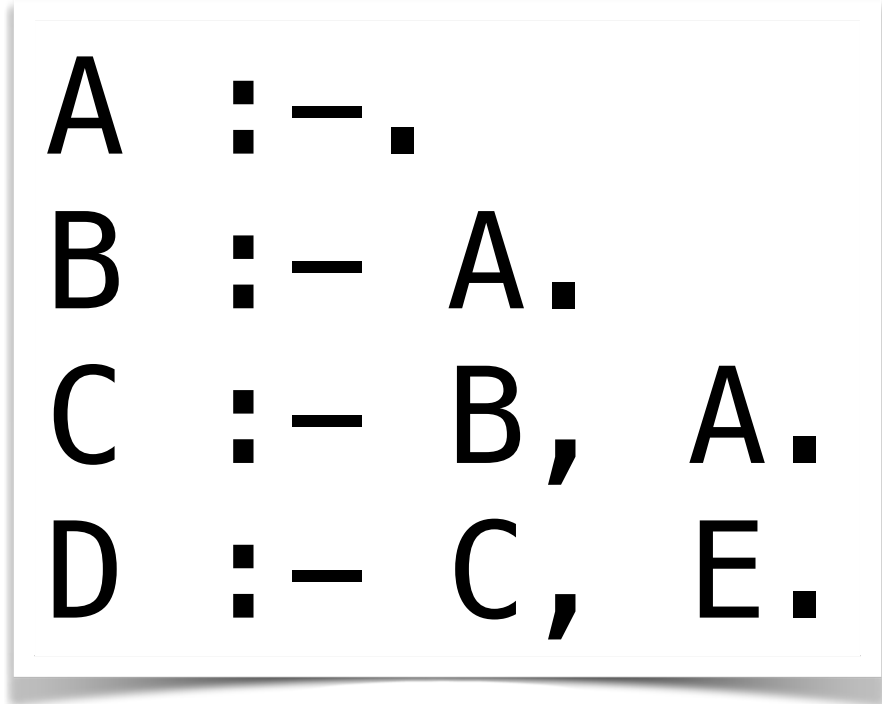
$$\text{se } B_1, B_2, \dots, B_n \subseteq F^i \text{ então } F^{i+1} := F^{i+1} \cup \{H\}$$
$$i := i + 1$$
$$\text{enquanto } F^{i+1} = F^i$$

Abstract representation of programs (simplified)

```
type term = string
type rule = term * term list
type program = rule list
type query = term
```

```
let program = [
  ("A", []);
  ("B", ["A"]);
  ("C", ["B"; "A"]);
  ("D", ["C"; "E"])
]
```

```
solve : program -> query -> bool
```



```
A :- .
B :- A.
C :- B, A.
D :- C, E.
```

Abstract representation of programs (simplified)

```
type parameter = Var of string | Value of int
type term = string * parameter list
type rule = term * term list
type program = rule list
type query = term
```

```
let program = [
  (( "A", [Value 0]), [])
; (( "B", [Var "X"]), [])
; (( "C", [Var "Y"]), [( "A", [Var "Y"]); ( "B", [Var "Y"])] )
; (( "D", [Var "Y"]), [( "B", [Var "Y"])] )
]
```

```
solve program ( "A", [Value 0] )
```

```
A(0) :- .
B(X) :- .
C(Y) :- A(Y), B(Y).
D(Y) :- B(Y).
```

```
A(0).
B(X).
C(0).
D(X).
```


Unification and Normalization of rules

$A(\emptyset) :- \cdot$

$B(X) :- \cdot$

$C(Y) :- A(Y), B(Y) \cdot$

$D(Y) :- B(Y) \cdot$

$?- C(X) \text{ (No)}$

$A(\emptyset) \cdot$

$B(X) \cdot$

$C(\emptyset) \cdot$

$D(X) \cdot$

Unification and Normalization of rules

$A(\emptyset) \text{ :- } \cdot$
 $B(X) \text{ :- } \cdot$
 $C(Y) \text{ :- } A(Y), B(Y) \cdot$
 $D(Y) \text{ :- } B(Y) \cdot$

$?- A(\emptyset) \text{ (Yes)}$

$A(\emptyset) \cdot$
 $B(X) \cdot$
 $C(\emptyset) \cdot$
 $D(X) \cdot$

Unification and Normalization of rules

$A(\emptyset) \text{ :- } \cdot$
 $B(X) \text{ :- } \cdot$
 $C(Y) \text{ :- } A(Y), B(Y) \cdot$
 $D(Y) \text{ :- } B(Y) \cdot$

$?- B(Y) \text{ (Yes ??)}$

$A(\emptyset) \cdot$
 $B(X) \cdot$
 $C(\emptyset) \cdot$
 $D(X) \cdot$

Unification and Normalization of rules

$A(_0, _0) :- .$
 $A(0, 1) :- .$
 $A(1, 2) :- .$
 $A(_0, _1) :- A(_0, W), A(W, _1)$

?- $A(0, 0)$ (Yes)
?- $A(1, 1)$ (Yes)
?- $A(8, 8)$ (Yes)
?- $A(0, 2)$ (Yes)
?- $A(Y, Y)$ (Yes)

Normalisation

```
let subst_of_head (f,ps) =  
  List.fold_left  
    (fun (i,sigma) -> function  
      | (Var v) ->  
        begin try  
          ignore(List.assoc v sigma); (i,sigma)  
        with Not_found -> (i+1,(v, Var ("_"^string_of_int i))::sigma)  
        end  
      | p -> (i,sigma))  
    (0,[]) ps
```

```
let normalize_variables (head, body) =  
  let (_,sigma) = subst_of_head head in  
  (apply_term head sigma, List.map (fun t -> apply_term t sigma) body)
```

$$A(X,Y) :- A(X,W), A(W,Y)$$
$$A(_0,_1) :- A(_0,W), A(W,_1)$$

Unification

```
let unify_term (f, ps) (f', ps') sigma =
  if f = f' then begin
    let rec unify_params ps ps' sigma =
      match ps, ps' with
      | [], [] -> Some sigma
      | p::ps, p'::ps' ->
        let sigma = unify_params ps ps' sigma in
        bind_maybe (unify_param p p') sigma
      | _ -> None
    in
    unify_params ps ps' sigma
  end
else None
```

$A(X, Y) - A(0, 1) \quad (X/0, Y/1)$

$A(X, Y) - A(W, W) \quad (X/W, Y/W)$

```
let unify_param p p' sigma =
  let p = apply sigma p in
  let p' = apply sigma p' in
  match p, p' with
  | Value v, Value v' when v = v' -> Some sigma
  | Var v, Var v' when v = v' -> Some sigma
  | Var v, Var v' -> Some ((v, Var v')::sigma)
  | Var v, Value v' -> Some ((v, Value v')::sigma)
  | Value v, Var v' -> Some ((v', Value v)::sigma)
  | _ -> None
```

Fixed point algorithm

- The (least) fixed-point algorithm allows the implementation of any inductive system (in polynomial time).

```
let solve program query =  
  let mem = Hashtbl.create 55 in  
  let rec flood () =  
    let changed = ref false in  
    List.iter (process_rule changed mem) program;  
    if !changed then flood ()  
  in  
  flood(); (eval_term mem query, terms_of mem) {H}
```

$i := i + 1$
enquanto $F^{i+1} = F^i$

Fixed point algorithm

- The (least) fixed-point algorithm allows the implementation of any inductive system (in polynomial time).

```
let rec process_rule changed mem (head, body) =  
  let (head, body) = normalize_variables (head, body) in  
  let substs = instances_of_terms body mem [] in  
  let apply_subst sigma =  
    match sigma with  
    | None -> ()  
    | Some sigma ->  
      let instance = apply_term head sigma in  
      add_instance changed mem instance  
  in  
  List.iter apply_subst substs
```


Fixed point algorithm

- The (least) fixed-point algorithm allows the implementation of any inductive system (in polynomial time).

```
let unify_term (f, ps) (f', ps') sigma =  
  let if f = f' then begin  
    let rec unify_params ps ps' sigma =  
      let match ps, ps' with  
      | [], [] -> Some sigma  
      | p::ps, p'::ps' ->  
        let sigma = unify_params ps ps' sigma in  
        bind_maybe (unify_param p p') sigma  
      | _ -> None  
    in  
    unify_params ps ps' sigma  
  end  
  else None
```

Fixed point algorithm

- The (least) fixed-point algorithm allows the implementation of any inductive system (in polynomial time).

```
let rec process_rule changed mem (head, body) =  
  let (head, body) = normalize_variables (head, body) in  
  let substs = instances_of_terms body mem [] in  
  let add_instance changed mem (f, ps) =  
    if not (Hashtbl.mem mem f) then Hashtbl.add mem f PSet.empty;  
    let l = Hashtbl.find mem f in  
    if not (PSet.mem ps l) then begin  
      Hashtbl.add mem f (PSet.add ps l); changed := true  
    end  
  in  
  List.iter apply_subst substs
```

Top-down search (sketch)

```
let rec solve (goal:term) (rules:rule list) =  
  let rec apply_rule (goal:term) (head,body) =  
    unify goal head  
    |>  
    fun s -> (List.fold_left (fun s t -> s @ solve_goal (apply_subst_term s t) rules) s body); s  
  and solve_goal (goal:term) (rules:rule list) =  
    match rules with  
    | [] -> raise Fail  
    | r::rules' -> try apply_rule goal r with Fail -> solve_goal goal rules'  
  and solve_goals s gs =  
    match gs with  
    | [] -> s  
    | g::gs' -> solve_goal g rules @ solve_goals s gs'  
  in solve_goals [] [goal]
```

- Only one solution is found...

Top-down search (sketch)

```
let prog : program = [  
  (* parent(v0,v1) :- *)  
  ( ("parent", [Value 0; Value 1]), [] );  
  ( ("parent", [Value 0; Value 2]), [] );  
  ( ("parent", [Value 1; Value 3]), [] );  
  ( ("parent", [Value 2; Value 4]), [] );  
  
  (* ancestor(X,Y) :- parent(X,Y) *)  
  ( ("ancestor", [Var "X"; Var "Y"]),  
    [ ("parent", [Var "X"; Var "Y"])] );  
  
  (* ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z) *)  
  ( ("ancestor", [Var "X"; Var "Z"]),  
    [ ("parent", [Var "X"; Var "Y"]);  
      ("ancestor", [Var "Y"; Var "Z"])] );  
]
```

```
let _ = solve ("ancestor", [Var "X"; Var "Y"]) prog  
let _ = solve ("ancestor", [Value 0; Value 2]) prog  
let _ = try ignore (solve ("ancestor", [Value 2; Value 1]) prog) with Fail -> ()  
let _ = solve ("ancestor", [Value 0; Value 3]) prog
```


Backtracking (prep, sketch - CPS)

```
let rec solve (goal:term) (rules:rule list) =  
  let rec apply_rule (goal:term) (head,body) k =  
    try  
      unify goal head  
      |>  
      fun s -> (List.fold_left (fun s t -> s @ solve_goal (apply_subst_term s t) rules k) s body)  
    with Fail -> k []  
  
  and solve_goal (goal:term) (rules:rule list) k =  
    match rules with  
    | [] -> k []  
    | r::rules' -> apply_rule goal r (fun _ -> solve_goal goal rules' k)  
  
  and solve_goals s gs k =  
    match gs with  
    | [] -> k []  
    | g::gs' -> solve_goal g rules (fun s -> solve_goals s gs' (fun s -> s @ k []))  
  
  in solve_goals [] [goal] Fun.id
```


Backtracking

- To get all possible solutions we need to implement backtracking

Backtracking (prep, sketch)

```
type subst = (string * parameter) list

let rec solve (goal:term) (rules:rule list) =

  let ks: ((subst -> subst) list) ref = ref [] in

  let rec apply_rule (goal:term) (head,body) k =
    try
      unify goal head |> fun s -> (List.fold_left (fun s t -> s @ solve_goal (apply_subst_term s t) rules k) s body)
    with Fail -> match !ks with
      | [] -> raise Fail
      | k'::ks' -> ks := ks'; k' []

  and solve_goal (goal:term) (rules:rule list) k =
    match rules with
    | [] -> k []
    | r::rules' -> ks := (fun _ -> solve_goal goal rules' k)::!ks; (apply_rule goal r k)

  and solve_goals s gs k =
    match gs with
    | [] -> k []
    | g::gs' -> solve_goal g rules (fun s -> solve_goals s gs' (fun s -> s @ k []))

  let rec let loop = if !ks = [] then [] else let k'::ks' = !ks in ks := ks'; k' []

  in ks := [fun _ -> solve_goals [] [goal] Fun.id]; loop ()
```

Prolog

- Prolog adds functors to possible parameters and arguments, making the language Turing complete and therefore without termination guarantees.
- Datalog always terminates, Prolog does not (it also has negation — failure).
- Datalog is evaluated bottom-up, Prolog runs on a depth-first top-down search.